



Programmazione II

Introduzione

Docenti

- Teoria B (cognomi E-O):
 - Prof. **Robert Birke** robert.birke@unito.it
 - Ricevimento su appuntamento
- Laboratorio:
 - Prof. **Marino Segnan** (B1: matricole dispari)
 - Prof. **Gianluca Torta** (B2: matricole pari)
- Tutorato:
 - Dott. **Francesco Medina** francesco.medina@unito.it

Corso



- **Programmazione 2: 6 CFU (150 ore)**
 - Ogni **C**redito **F**ormativo **U**niversitario (**CFU**) sono 25 ore di lavoro studente
- In aula:
 - Teoria: 16 lezioni da 2h (32 ore)
 - Laboratorio: 10 esercitazioni da 2h (20 ore)
- Restanti 98 ore?
 - Servono per studiare ed esercitarsi **in autonomia**

Lezioni

- Programma più **intensivo** rispetto a Programmazione I
- Lezioni comprendono anche **esercizi**
 - Gli esercizi vi faranno lavorare a coppie/gruppi
- Le **domande** sono **benvenute**

Laboratorio

- **Esercizi** assegnati dai docenti di laboratorio
 - Iniziati in laboratorio ...
 - ... e da **completare in autonomia** a casa
- Soluzioni da **consegnare su Moodle**
- Devono superare:
 - i **test automatici** (non esaustivi)
 - l'**approvazione del docente** del proprio turno

... e gli Esami



- Spiegatele alla Nonna*:
 - “You do not really understand something unless you can explain it to your grandmother.”
— **Albert Einstein**
 - Studiate finché il **discorso** non diventa **fluido** e le **connessioni** fra gli argomenti **chiare**
- Imparare **facendo** (Learning by doing):
 - Per imparare a programmare serve anche **esercitarsi**: prima su carta e poi al computer perché dovete **prima formare la mente** a identificare procedimenti e strutture dati
- **Confronto**:
 - Scambiatevi soluzioni, commentatele a vicenda, fatevi delle domande

Errori Comuni

- Programmare $\neq$ usare computer
 - Si può programmare con carta e penna
 - Computer può essere un falso amico: si sostituisce a voi riducendo il vostro apprendimento
- Andare a tentativi:
 - Scrivere codice **quasi a caso**, guidati dalle segnalazioni di errore del compilatore
 - Bisogna avere le idee chiare delle istruzioni impartite al computer

Metafora del Interprete*

English is either the official language or one of the official languages in 59 sovereign states (such as in India, Ireland, and Canada). In some other countries, it is the sole or dominant language for historical reasons without being explicitly defined by law (such as in the United States or United Kingdom)

- Supponiamo che un interprete traduca un brano come alcuni usano il compilatore:
 - Raffazzona un primo tentativo
 - Aspetta le domande di chi perplesso non capisce
 - Prova ad aggiustare il tiro

Prima Traduzione

English is either the official language or one of the official languages in 59 sovereign states (such as in India, Ireland, and Canada). In some other countries, it is the sole or dominant language for historical reasons without being explicitly defined by law (such as in the United States or United Kingdom)

A green arrow pointing from the left text to the right text, with the words "Prima Traduzione" written inside it.

Prima Traduzione

In alcuni altri paesi in 59 stati sovrani per ragioni storiche è il solo, l'inglese o una delle lingue ufficiali è la lingua ufficiale (come gli Stati Uniti o il Regno Unito) (come in India, Irlanda e Canada in 59 stati sovrani). Senza essere esplicitamente definito per legge dominante.

È comprensibile?

Tentativi con Raffinamenti Successivi

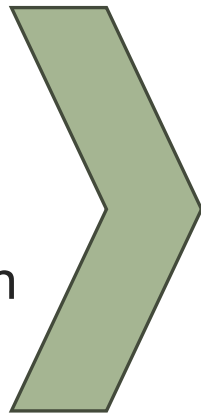
- Difetti dell'approccio:
 - Lavoro alla cieca, spesso con tentativi semi-casuali, non organizzati
 - Si applicano pattern memorizzati ma non veramente capiti
 - Il programmatore non controlla il programma, non ha certezza che sia corretto e completo
 - Si spreca un sacco di tempo
 - Si costruiscono soluzioni inefficienti
- Scrivere codice direttamente al computer supervisionati dal compilatore:
 - Esempio di procedimento per raffinamenti successivi
- L'idea dei raffinamenti successivi è valida
 - Il modo di procedere non lo è

Esame



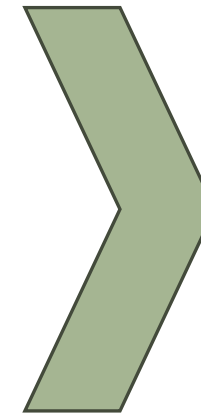
Esercizi

- Prerequisito per l'esame:
- Aver **svolto e consegnato** su **Moodle** tutti gli esercizi assegnati in laboratorio
- Superano i **test automatici**
- **Approvati** dal **docente** di laboratorio



Laboratorio [0-3 punti]

- Correzione immediata davanti al candidato
 - superata se voto ≥ 1
- Vale per i 5 appelli successivi a quello in cui è stata superata
 - incluso quello della prova



Teoria [0-30 punti]

- Domande aperte
 - Esercizi
- ... e chiuse
 - Vero/falso
 - Scelta multipla
- Corretta nei giorni successivi
 - superata se voto ≥ 17

Esame

- Voto finale:
 - voto di teoria (≥ 17) + voto di laboratorio (≥ 1)
 - se ≥ 31 viene verbalizzato come 30 e Lode



Macro-Argomenti Trattati

- Ripasso concetti di base di programmazione e C
- Programmi suddivisi su più file e riutilizzo del codice
- Gestione dinamica della memoria
- Strutture dati dinamiche lineari
- Tipi di dati composti e astratti
- Lettura e scrittura formattata su file
- Metodologia di programmazione

Macro-Obiettivo del Corso

- Obiettivo del corso: insegnarvi a programmare!
- Linguaggi (di programmazione e non) modellano il pensiero:

”A programming language is not just – or even primarily – a means for programmers to communicate with computers: it is also a means for programmers to communicate with programmers – including themselves. It is a social, as well as a technical, enabler.” – Andrew P. Black
- È importante:
 - “Vedere la programmazione come metodo per descrivere il mondo a un calcolatore, indipendentemente dal linguaggio di programmazione.” – Luca Roversi
 - Ponendo enfasi: “su concetti chiave, il più possibile indipendenti dal linguaggio di programmazione.” – Luca Roversi

Linguaggio C: è lo strumento usato ma molti concetti sono universali

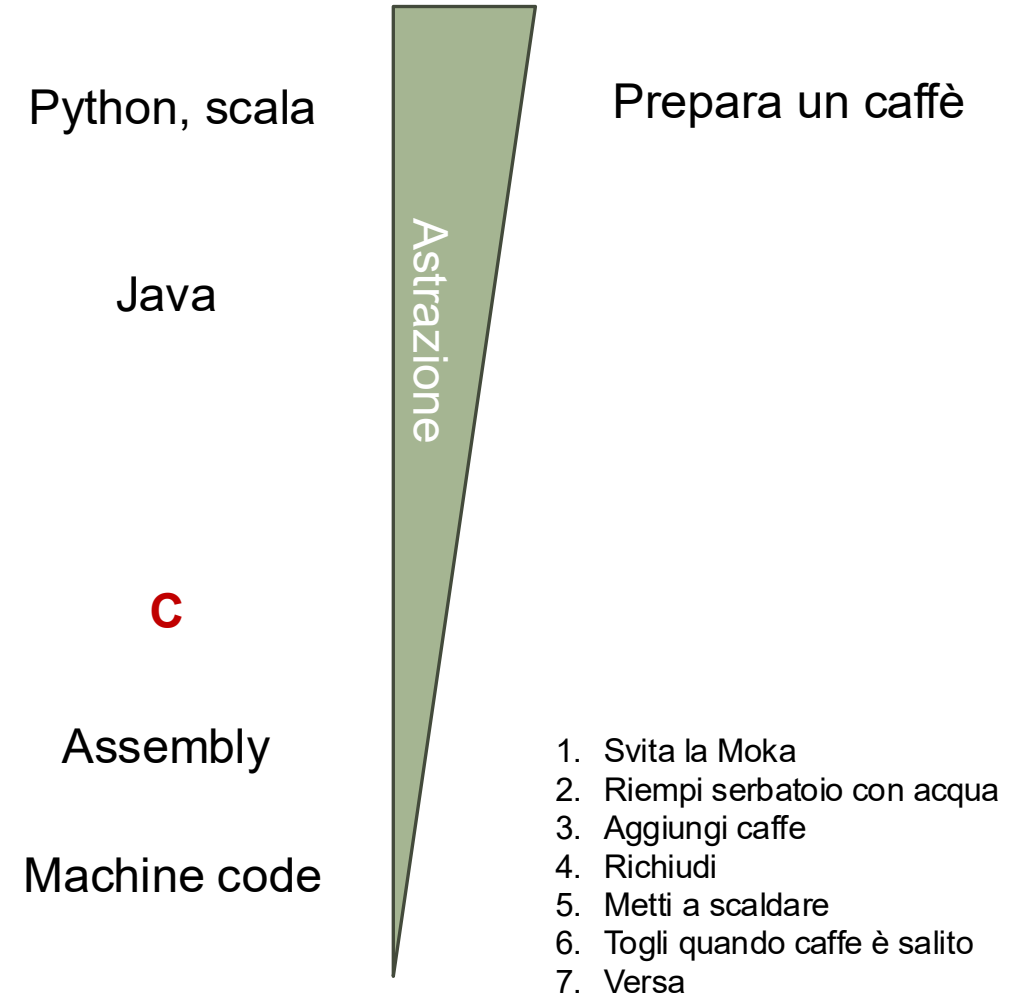
Linguaggio C

- Linguaggio C:
 - Nato per riscrivere il sistema operativo Unix*
 - Dichiarazioni di tipo servono a specificare quanto spazio di memoria occupa una variabile
- Ancora in evoluzione:
 - Ultimo standard C24 del 2024
 - <https://blog.ansi.org/ansi/c-programming-language-standard-iso-iec-9899-2024/>
 - Al fine del corso indistinguibile da C11 e C18

*Turing Award 1983: Dennis Ritchie e Ken Thompson per lo sviluppo di UNIX

Perché C? I

- C ampiamente disponibile e indipendente dall'hardware
 - Applicazioni C spesso supportate su un'ampia gamma di sistemi informatici
- Linguaggio di astrazione media
 - Non nasconde troppi dettagli
 - Permette di capire che cosa succede "sotto il cofano»
 - Iniziare con C permette di apprezzare meglio i pregi, difetti e limiti dei linguaggi con astrazioni di livello più alti



Perché C? II

- C è molto usato
 - Alcune applicazioni richiedono esclusivamente (o quasi) il C
 - Esempi: sistemi operativi, embedded
- C è «small and simple»
 - Ma non facile
 - Permette di focalizzarsi su pochi aspetti
- Codificare in C richiede molta attenzione
 - Se imparate a cavarvela in C, sarà facile con (quasi tutti) gli altri linguaggi

Moodle del Corso

- Iscrivetevi ai Moodle del corso!
- Moodle usato per (non esaustivo):
 - Distribuzione materiale
 - Lucidi, esercizi
 - Consegna dei progetti
 - Pubblicazione avvisi
 - Esempio: cambiamento orari

Teoria B
ProgII-B [Cogn.
E-O] (aa 25/26)

Cognomi E-O

Laboratorio B1
ProgII-LabB1

Matricole dispari

Laboratorio B2
ProgII-LabB2

Matricole dipari

Libro di Testo

“P. J. Deitel & H. M. Deitel, Il linguaggio C - Fondamenti e tecniche di programmazione, Pearson, 2022”.

- Riferimento principale per dettagli omessi
 - ... ma non copre tutti gli argomenti
- Stesso di Programmazione I
 - ... dovrete sapere gli argomenti dei capitoli 1-8

C How to Program, Ninth Edition
with Case Studies Introducing Applications Programming and Systems Programming
by Paul Deitel & Harvey Deitel

PART 1 (Introductory) **Programming Fundamentals** **Quickstart**

1. Introduction to Computers and C
Intro to Hardware, Software & Internet; Test-Drive Microsoft Visual Studio, Apple Xcode, GNU gcc & GNU gcc in Docker

2. Intro to C Programming
Input, Output, Types, Arithmetic, Decision Making, Secure C

3. Structured Program Development
Algorithm Development, Problem Solving, if, if/else, while, Secure C

4. Program Control
for, do/while, switch, break, continue, Logical Operators, Secure C

5. Functions
Custom Functions, Simulation, Random-Number Generation, Enumerations, Function Call and Return Mechanism, Recursion, Recursive Factorial, Recursive Fibonacci, Secure C

- C is one of the world's most popular and senior programming languages
- C18/C11 standards
- Topical, innovative presentation
- Rich coverage of fundamentals
- Problem-solving/developing algorithms
- 20+ fun computer-science, data-science and artificial-intelligence case studies show C as it's intended to be used—some are fully implemented, some are partially implemented and some require students to do online research
- 147 complete working programs
- 350+ integrated self-check exercises with answers
- 445 end-of-chapter exercises/projects
- Use with Windows®, macOS®, Linux®
- Visual C++®, Xcode® and GNU™ gcc

PART 2 (Intermediate) **Arrays, Pointers** **and Strings**

6. Arrays
One- & Two-Dimensional Arrays, Passing Arrays to Functions, Searching, Binary Search Visualization, Sorting, Secure C

7. Pointers
Pointer operators & and *, Pass-By-Value vs. Pass-By-Reference, Array and Pointer Relationship, Secure C

8. Characters and Strings
C Standard Library String- and Character-Processing Functions, Secure C

PART 3 (Intermediate) **Formatted Input/Output, Structs** **and File Processing**

9. Formatted Input/Output
scanf and printf formatting, Secure C

10. Structures, Unions, Bit Manipulation and Enumerations
Creating Custom Types with structs and unions, Bitwise Operators, Enumeration Constants, Secure C

11. File Processing
Streams, Text and Binary Files, CSV Files, Sequential and Random-Access Files, Secure C

- Analysis of algorithms with Big O
- Enhanced security and data science coverage as per ACM/IEEE 2020 curricula recommendations
- Use free open-source libraries and tools
- Real-world examples and data
- Traditional or “flipped” classrooms
- Secure C Programming, privacy, ethics
- Case studies in systems programming and applications programming
- Think like a developer with GitHub®, open-source, StackOverflow and more

PART 4 (Advanced) **Data Structures and** **Algorithms**

12. Data Structures
Dynamic Memory Allocation, Lists, Stacks, Queues & Binary Trees, Secure C

13. Computer-Science Thinking: Sorting Algorithms and Big O
Insertion Sort, Selection Sort, Visualizing Merge Sort, Additional Algorithms including Quicksort in the Exercises

PART 5 (Advanced) **Preprocessor and Other Topics**

14. Preprocessor
#include, Conditional Compilation, Macros/Arguments, Assertions, Secure C

15. Other Topics
Variable-Length Argument Lists, Command-Line Arguments, Multiple-Source-File Programs, extern, exit/atexit, calloc/realloc, goto, Numeric Literal Suffixes, Signal Handling

Appendices

A. Operator Precedence
B. ASCII Character Set
C. Multithreading/Multicore and Other C11/C18 Topics
D. Intro to Object-Oriented Programming

Online Appendices

E. Number Systems
F–H. Using the Visual Studio, GNU gdb and Xcode Debuggers

- Emphasis on visualization
- Static code analysis tools
- Performance, multithreading, multicore
- Questions? deitel@deitel.com
- Updates and errata: <https://deitel.com/cht9>

Systems Programming **Case Studies**

Systems Software

- Building Your Own Computer
- Building Your Own Compiler with Infix and Postfix Notation

Embedded Systems Programming

- Webots 3D Robotics Simulator

Performance: Threading/Multicore

Applications Programming **Case Studies**

Algorithm Development

- Counter-Controlled Iteration
- Sentinel-Controlled Iteration
- Nested Control Statements

Random-Number Simulation

- Building a Casino Game

- Card Shuffling/Dealing with Card Images
- The Tortoise and the Hare Race

Intro to Data Science

- Data Analysis: Mean, Median & Mode

Direct-Access File Processing

- Transaction-Processing System

Visualizing Searching & Sorting**Artificial Intelligence/Data Science**

- Machine Learning, GNU Scientific Library, Plotting with gnuplot, CSV Files
- NLP: Who Wrote Shakespeare's Works?

Game Programming with raylib

- SpotOn and Cannon Games

Security Via Cryptography

- Secret-Key & RSA Public-Key Crypto

Visualization with raylib

- Law of Large Numbers Animation

Multimedia: Audio & Animation**Web Services, Mashups, Cloud**

- Accessing Web Services with libcurl; OpenWeatherMap JSON Results
- Rapid Applications Development with Web-Service Mashups



II «Kernighan-Ritchie»

- Deitel: insegnare a programmare usando il C
- Kernighan-Ritchie: insegnare il linguaggio C

